

11. 정보지원팀 — 전산장애

공유 게이트웨이(:11500) 위에서 내 앱을 운영급으로. 이 과제는 코드 제출본이 없어, “어떤 앱이든 게이트웨이에 붙이는 표준 방법”으로 안내합니다. 주소 한 줄(:11434 → :11500)만 바꾸면 끝입니다.

1) 내 앱은 이런 모양 (A RAG 표준 구조)

전산장애·업무 안내 챗봇은 **근거기반(RAG) 챗봇**입니다. 정보지원팀 안내 자료(외부영상CD 처리, 물품 계약·보증 규정 등)를 미리 조각으로 넣어 두고, 질문과 **가장 가까운 조각만 골라** 출처와 함께 답하는 구조입니다.



- **프론트(index.html)**: 질문을 입력받고 답·출처를 그립니다. **모델을 직접 부르지 않습니다**. 오직 내 백엔드(:8000)만 호출합니다.
- **내 백엔드(app.py , 포트 8000)**: 자료를 검색하고, 보증금 같은 숫자는 코드로 정확히 계산한 뒤, 모델에게 답 생성을 맡깁니다.
- **공유 게이트웨이(:11500)**: 모든 학생이 함께 쓰는 모델 서버. 임베딩(검색)과 채팅(답변)을 여기서 처리합니다.

핵심 원칙은 “**브라우저는 백엔드만, 모델 호출은 백엔드가**” 입니다. 화면(HTML)이 모델을 직접 부르면 보안·CORS 사고가 납니다.

참고 자료: 이 챗봇에 넣을 정보지원팀 안내 조각은 **전산장애대응_자료정제.md**에 정리되어 있습니다(외부영상CD 처리 원칙, 보증보험 5/100·10/100 규정 등). 그대로 **app.py**의 **DOCS** 리스트에 넣으면 됩니다.

2) 바뀌는 것 — URL 한 줄

운영급으로 가는 변경은 **딱 한 줄**입니다. 백엔드 코드에서 모델 주소를 게이트웨이(:11500)로 두기만 하면 됩니다.

```
# app.py 상단 — 모델 주소를 한 곳에 모아 둡니다
OLLAMA_URL = "http://121.138.151.6:11500"
```

이 한 줄이 검색(임베딩)·생존확인·채팅 세 가지 호출 전부를 한꺼번에 제어합니다. 아래처럼 이 변수만 갖다 쓰면 주소를 바꿀 곳이 여기 하나뿐입니다.

```
# 임베딩(검색) → f"{OLLAMA_URL}/api/embeddings"
# 생존 확인 → f"{OLLAMA_URL}/api/tags"
# 채팅(답변) → f"{OLLAMA_URL}/api/chat"
```

예전에는 각자 띄우던 raw 주소 :11434 를 썼습니다. 거기서는 모델의 혼잣말(생각)이 답에 섞여 나오거나, 여럿이 몰리면 빈 답이 나오곤 했습니다. 포트 한 글자만 바꾸면 해결됩니다.

```
- OLLAMA_URL = "http://121.138.151.6:11434" # 예전: raw 주소(생각 누출·부하 위험)
+ OLLAMA_URL = "http://121.138.151.6:11500" # 지금: 공유 게이트웨이(자동 보정)
```

게이트웨이가 알아서 해주는 것: ① `think:false` (생각 누출 차단) ② `num_predict` 상한(응답 시간 안정) ③ 모델명 정규화(이름이 조금 달라도 맞춰줌). **학생은 신경 쓸 필요 없습니다 — 주소만 :11500이면 됩니다.**

3) 확인 절차

(1) 게이트웨이가 살아있나 — `curl` 로 점검

```
curl http://121.138.151.6:11500/api/tags
```

모델 목록(JSON)이 돌아오면 게이트웨이 정상입니다.

(2) 앱 기동

내 앱 폴더로 가서 백엔드를 띄웁니다(포트 8000 고정 — 프론트가 8000을 바라봅니다).

```
uvicorn app:app --port 8000
```

그다음 앱 자체가 살아있는지 `/health` 로 확인합니다.

```
curl http://127.0.0.1:8000/health
# 예: {"ok":true,"ollama":true,"model":"qwen3.6:35b-a3b","docs":13,...}
```

`"ollama":true` 면 백엔드에서 게이트웨이까지 연결 정상입니다. 그다음 프론트를 띄웁니다 — `index.html` 을 파일로 직접 더블클릭하면 CORS로 막힐 수 있으니, 같은 폴더에서 `python -m http.server 5500` 을 실행하고 브라우저에서 `http://127.0.0.1:5500/index.html` 을 여세요. 화면 상태بات지가 초록(정상)이 되는지 보세요.

(3) 대표 질문 1개 — 한번 돌려보기

화면 입력창에 아래를 넣어 보세요.

CD 분석이 안 돼요

기대 동작: 답 위/아래에 **출처 태그**(예:  외부영상CD-CD분석중) 가 뜨고, “CD-ROM 교체, 담당은 정보지원팀” 같은 근거 답이 흘러나옵니다. 명령줄에서 바로 확인하려면:

```
curl -N -X POST http://127.0.0.1:8000/api/chat \
-H "Content-Type: application/json" \
-d '{"question":"CD 분석이 안 돼요"}'
```

(4) think(생각) 누출 없는지 확인

답변 안에 `<think>` 같은 태그나 “사용자가 ~을 물었으니 먼저...” 같은 **모델의 혼잣말**이 섞이면 안 됩니다. 게이트웨이가 `think:false` 를 자동으로 걸어주므로 깔끔한 답만 나오는 것이 정상입니다. 만약 혼잣말이 보이면 모델 주소가 `:11500` 인지부터 다시 확인하세요 (`:11434` 로 남아 있으면 누출됩니다).

4) 아직 코드가 없다면 — 3주차 STEP 0으로 출발선 맞추기

이 과제는 제출 코드가 없으니, 먼저 **3주차에서 만들던 분리 챗봇(프론트 + 백엔드)** 을 띄워 출발선을 맞춥니다. 3주차 개인 핸드아웃에 **STEP 0 캐치업**이 그대로 들어 있습니다.

- 핸드아웃: [lecture/3주차_강의_개인별/3주차_강의_11_전산장애.pdf](#)

STEP 0 요약(이대로 따라 하면 됩니다):

1. 공통 핸드아웃의 **STEP 0 기본코드**를 받아 `app.py` (FastAPI 백엔드, :8000)와 `index.html` 을 띄운다.
2. 백엔드의 모델 주소를 **게이트웨이 :11500** 으로 둔다(위 §2의 한 줄). 3주차에는 `:11434` 였으니 이번 주에는 **11500** 으로 바꾸는 것이 핵심입니다.
3. 터미널에서 `curl http://127.0.0.1:8000/health` 가 200으로 응답하는지 확인.
4. 브라우저에서 `index.html` 을 열어  **연결됨** 표시와 함께 질문 1개에 답이 오면 출발 준비 완료.
5. 그다음 정보지원팀 자료 조각([전산장애대응_자료정제.md](#) 의 13조각)을 `app.py` 의 `DOCS` 리스트에 넣어 검색(RAG)을 붙입니다.

✅ **출발선 성공 기준:** `/health` 가 200을 주고, “CD 분석이 안 돼요” 질문에 답변 텍스트가 화면에 뜨면 OK. 여기까지 되면 §1~§3을 그대로 따라 운영급으로 올릴 수 있습니다.

5) 안전수칙 (정보지원팀 특화)

- **더미·비식별 데이터만** — 자료 조각·테스트 질문은 모두 연습용·비식별만 씁니다. 실제 사례·내부 통계·담당자 실명/연락처는 넣지 마세요.
- **환자 개인정보·실제 보안정보 절대 금지** — 환자명·등록번호는 물론, **서버 비밀번호·관리자 계정·상세 IP 대역** 같은 보안정보도 입력 금지입니다. 챗봇은 업무 안내 범위까지만. (그런 질문이 오면 모델 호출 없이 “보안상 안내하지 않습니다”로 차단하도록 만들면 좋습니다.)
- **분리 강제** — 화면(`index.html`)에서 모델(`:11434 / ollama`)을 직접 부르지 마세요. 모델 호출은 오직 백엔드(:8000) 경유. 이 원칙은 보안·CORS 사고를 막는 기본입니다.

- **공유 GPU는 1개, 동시성 주의** — 게이트웨이 뒤 GPU는 모두가 함께 씁니다. 여러 명이 동시에 긴 질문을 던지면 느려집니다. 실습 때는 한 번에 한 질문씩, 답이 끝나면 다음 질문을.
- **공유 서버 = 운영 책임** — 인증 없는 공유 환경입니다. 테스트가 끝나면 띄운 `uvicorn` 을 내려(Ctrl+C) 불필요한 부하를 남기지 마세요. 모델 서버 주소(`:11500`)·접속법은 외부 공유 금지.